

Lecture: Quantitative Variables tut

By. Dr. Mohammed Khalaf

- Part I: Multiple Choice Questions
- Part II: Expected Short Answer Questions
- Part III: Multiple Choice Answer Key and Explanations
- Part IV: Code Reference Sheet

- Loading and Exploring Data
- Working with a Single Quantitative Variable
- Visualization
- Manual Binning with `pd.cut()`
- Custom Statistical Functions (First Principles)
- Formatting Output

Lecture: Quantitative Variables tut

By. Dr. Mohammed Khalaf

Part I: Multiple Choice Questions

Instructions: For each question, select the best answer. Code snippets are provided where relevant. Answers and explanations are provided at the end of this document.

1. A data scientist has loaded a Pandas DataFrame `df` containing a column `'age'` . They want to create a histogram to examine the distribution of ages. Which line of code will accomplish this?

- a. `df['age'].plot.bar()`
- b. `df['age'].plot.hist()`
- c. `df['age'].value_counts().plot.pie()`
- d. `df['age'].plot.line()`

2. Consider the following Python code:

```
df['age_group'] = pd.cut(df['Edad'],
                        bins=[0, 18, 35, 50, 65, 120],
                        labels=['0-17', '18-34', '35-49', '50-64', '65+'],
                        right=False)
df['age_group'].value_counts(sort=False).plot.bar()
```

What is the primary purpose of using `pd.cut()` in this context?

- a. To remove missing age values from the dataset
- b. To create custom age bins that align with clinically meaningful categories before visualization
- c. To calculate the mean age more accurately
- d. To sort the DataFrame by age in ascending order

3. A data scientist runs the following code:

```
print(f"Mean: {df['age'].mean():.1f}")
print(f"Median: {df['age'].median():.1f}")
```

Output:

Mean: 39.0
Median: 37.0

Based on this output, what can you conclude about the shape of the age distribution?

- a. The distribution is perfectly symmetric
- b. The distribution is left-skewed (negatively skewed)
- c. The distribution is right-skewed (positively skewed)
- d. The distribution is bimodal

4. Which of the following code snippets correctly calculates the variance of a Pandas Series `df['age']` ?

- a. `df['age'].var()`
- b. `df['age'].std() ** 2`
- c. `((df['age'] - df['age'].mean()) ** 2).sum() / (len(df['age']) - 1)`
- d. All of the above

5. The standard deviation of a dataset is 15.2 years, and the mean is 42.5 years. Assuming an approximately normal distribution, between what two values would you expect roughly 68% of the data to fall?

- a. 27.3 and 57.7 years
- b. 12.1 and 72.9 years
- c. 42.5 and 57.7 years
- d. 0 and 42.5 years

6. A student writes the following code to calculate the median manually:

```
def custom_median(x):  
    sorted_x = sorted(x)  
    n = len(sorted_x)  
    if n % 2 == 1:  
        return sorted_x[n // 2]  
    else:  
        return (sorted_x[n // 2 - 1] + sorted_x[n // 2]) / 2
```

For an even-numbered dataset with sorted values [10, 20, 30, 40], what will this function return?

- a. 20
- b. 25
- c. 30
- d. 35

7. Which of the following statements accurately describes a key difference between a histogram and a bar chart?

- a. Histograms display categorical data, while bar charts display quantitative data
- b. Histograms have spaces between bars to emphasize discrete categories, while bar charts have no spaces
- c. Histograms have no spaces between bars to emphasize the continuous nature of the underlying variable, while bar charts have spaces to emphasize distinct categories
- d. Histograms are always vertical, while bar charts can be horizontal or vertical

8. The following code produces an error:

```
df['age'].value_counts().plot.hist()
```

Why does this code fail to produce a meaningful histogram of the original age data?

- a. The `.value_counts()` method converts the continuous age data into a categorical frequency table, so `.plot.hist()` is now being applied to aggregated counts rather than the original observations
- b. The syntax should be `df.plot.hist('age')` instead

- c. The `.value_counts()` method drops missing values, making the histogram invalid
- d. Histograms cannot be created from Pandas Series objects

9. A public health official reports: “The average age of COVID-19 cases in our city is 38 years.” A data scientist critiques this statement because:

- a. The mean alone provides no information about the variability or spread of ages across cases
- b. The median would always be a better measure for public health reporting
- c. The mode is the only acceptable measure for age data
- d. Averages cannot be calculated for health data

10. Which line of code correctly calculates the standard deviation of the ‘Edad’ column and prints it with a descriptive label?

- a. `print("Standard deviation:", df['Edad'].std())`
- b. `print("Standard deviation: " + str(df['Edad'].std()))`
- c. `print(f"Standard deviation: {df['Edad'].std():.2f} years")`
- d. All of the above produce valid output, but (c) is preferred for its formatting and inclusion of units

Part II: Expected Short Answer Questions

Instructions: Answer each question concisely but completely. Include relevant code snippets where requested. Demonstrate understanding of both statistical concepts and their implementation in Python.

Question 1: Visualizing Quantitative Data

A researcher provides you with a Pandas DataFrame `df` containing a column `'patient_age'` with 10,000 observations. They want to understand the distribution of ages to identify whether certain age groups are disproportionately affected by a disease.

(a) Write Python code to generate a histogram of the patient ages. Include a title and axis labels.

```
import matplotlib.pyplot as plt

# Generate histogram
df['patient_age'].plot.hist(bins=30, edgecolor='black', alpha=0.7)
plt.title('Distribution of Patient Ages', fontsize=14)
plt.xlabel('Age (years)')
plt.ylabel('Frequency')
plt.grid(axis='y', alpha=0.3)
plt.show()
```

(b) Explain why a histogram is more appropriate than a bar chart for this task.

Answer: A histogram is appropriate because `patient_age` is a quantitative (continuous) variable. Histograms group data into continuous bins along a number line, revealing the distribution’s shape, central tendency, and spread. Bar charts are designed for categorical data, where each bar represents a distinct, non-continuous category. Using a bar chart for continuous age data would treat each unique age as a separate category, losing the sense of continuity and making the visualization cluttered and uninformative for 10,000 observations.

(c) The researcher asks, “Why are there no gaps between the bars in your histogram?” How would you explain this in plain language?

Answer: “The lack of gaps is intentional. It visually communicates that age is a continuous scale—there’s no sharp break between someone who is 29 and someone who is 30. The bars touch because they represent consecutive intervals along the same number line. If I added gaps, it would incorrectly suggest that these age groups are separate categories, like ‘male’ and ‘female’.”

Question 2: Measures of Central Tendency

You have calculated the mean and median age for COVID-19 cases in two different regions:

Region	Mean Age	Median Age
A	42.3	41.8
B	42.3	28.5

(a) Write Python code to calculate and print the mean and median for a column named `'age'` in a DataFrame `df_region` .

```
mean_age = df_region['age'].mean()
median_age = df_region['age'].median()

print(f"Mean age: {mean_age:.1f} years")
print(f"Median age: {median_age:.1f} years")
```

(b) Both regions have the same mean age. What does this tell you? What does it NOT tell you?

Answer: The identical mean tells me that the arithmetic average age is the same in both regions. However, it does NOT tell me anything about the distribution’s shape, spread, or whether this average is representative of a typical case. The mean is sensitive to extreme values and can be misleading if the data is skewed.

(c) The median ages differ dramatically between the two regions. What does this reveal about the underlying age distributions?

Answer: The near-equality of mean and median in Region A (42.3 vs. 41.8) suggests a roughly symmetric distribution, where the average age is representative of a typical case. In Region B, the median (28.5) being substantially lower than the mean (42.3) indicates strong right-skewness. This means that while the “typical” (median) patient is around 28-29 years old, there is a tail of much older individuals pulling the average upward. Region B likely has a large young adult population affected, plus a smaller but significant group of elderly cases.

(d) As a public health official, how would your resource allocation strategy differ between these two regions?

Answer: In Region A, with a symmetric distribution around 42 years, resources should be broadly distributed across middle-aged adults, with standard hospital preparedness. In Region B, the low median suggests a large young adult caseload, warranting messaging targeting workplaces, universities, and social gatherings. However, the higher mean signals a non-negligible elderly population in the tail, requiring targeted protections for nursing homes and geriatric facilities. Region B needs a two-pronged strategy; Region A needs a more uniform approach.

Question 3: Measures of Spread

A data analyst reports that for a dataset of patient ages, the mean is 45.2 years and the standard deviation is 3.1 years.

(a) Write Python code to calculate both the variance and the standard deviation for a column 'age' . Show two different ways to obtain the standard deviation.

```
# Method 1: Direct standard deviation
std_dev1 = df['age'].std()

# Method 2: Standard deviation from variance
variance = df['age'].var()
std_dev2 = variance ** 0.5

print(f"Variance: {variance:.2f} years²")
print(f"Standard deviation: {std_dev1:.2f} years")
print(f"Standard deviation (from variance): {std_dev2:.2f} years")
```

(b) Explain the relationship between variance and standard deviation. Why is the standard deviation often preferred for reporting?

Answer: Variance is the average squared deviation from the mean. It is mathematically convenient but measured in squared units (e.g., years²), which are difficult to interpret intuitively. Standard deviation is the square root of the variance, returning the measure to the original units (e.g., years). This allows for intuitive statements like “the typical deviation from the average age is about 3.1 years,” which stakeholders can immediately understand.

(c) For the reported values (mean = 45.2, standard deviation = 3.1), what can you infer about the range where most patients’ ages likely fall? State any assumptions you make.

Answer: Assuming the age distribution is approximately normal (or at least not heavily skewed), we can apply the empirical rule. Approximately 68% of patients would fall within one standard deviation of the mean: between 42.1 years (45.2 - 3.1) and 48.3 years (45.2 + 3.1). Approximately 95% would fall within two standard deviations: between 39.0 years and 51.4 years. This suggests relatively low age variability, indicating the disease disproportionately affects a narrow middle-aged cohort.

(d) If the standard deviation were 15.2 years instead of 3.1 years, how would your interpretation change?

Answer: A standard deviation of 15.2 years would indicate much greater age variability. The 68% range would span from 30.0 to 60.4 years—a wide swath of the adult population. This would suggest the disease affects all age groups broadly, requiring a universal public health messaging strategy rather than targeted interventions. Healthcare systems would need to prepare for patients across the entire age spectrum, from young adults to the elderly.

Question 4: Implementing Statistical Functions

You are asked to demonstrate your understanding of statistical calculations by implementing functions from first principles, then comparing them to built-in methods.

(a) Write a Python function called calculate_mean that takes a list of numbers and returns the mean. Do not use built-in statistical functions like np.mean() or pd.Series.mean() .

```
def calculate_mean(data):
    """
    Calculate the arithmetic mean of a list of numbers.

    Parameters:
    data (list): A list of numeric values

    Returns:
    float: The mean of the input data
    """
    if len(data) == 0:
        return 0

    total = 0
    for value in data:
        total += value

    return total / len(data)
```

(b) Write a function called `calculate_median` that takes a list of numbers and returns the median. Handle both odd and even-length lists correctly.

```
def calculate_median(data):
    """
    Calculate the median of a list of numbers.

    Parameters:
    data (list): A list of numeric values

    Returns:
    float: The median of the input data
    """
    if len(data) == 0:
        return 0

    sorted_data = sorted(data)
    n = len(sorted_data)

    # If odd, return middle value
    if n % 2 == 1:
        return sorted_data[n // 2]
    # If even, return mean of two middle values
    else:
        left_mid = sorted_data[n // 2 - 1]
        right_mid = sorted_data[n // 2]
        return (left_mid + right_mid) / 2
```

(c) Write a function called `calculate_std` that takes a list of numbers and returns the sample standard deviation. Use your `calculate_mean` function. Do not use built-in `.std()` methods.

```
def calculate_std(data):
    """
    Calculate the sample standard deviation of a list of numbers.

    Parameters:
    data (list): A list of numeric values

    Returns:
    float: The sample standard deviation of the input data
    """
    if len(data) < 2:
        return 0 # Standard deviation undefined for samples of size 1

    mean = calculate_mean(data)
    squared_deviations = 0

    for value in data:
```

```
squared_deviations += (value - mean) ** 2

variance = squared_deviations / (len(data) - 1) # Sample variance (n-1)
return variance ** 0.5
```

(d) Test your functions on the list `ages = [23, 45, 67, 34, 56, 78, 45, 34, 56, 67]` and compare the results to Pandas' built-in methods. Write code to demonstrate they produce the same results.

```
import pandas as pd

# Test data
ages = [23, 45, 67, 34, 56, 78, 45, 34, 56, 67]

# Custom functions
custom_mean = calculate_mean(ages)
custom_median = calculate_median(ages)
custom_std = calculate_std(ages)

# Pandas built-in methods
df_test = pd.DataFrame({'age': ages})
pandas_mean = df_test['age'].mean()
pandas_median = df_test['age'].median()
pandas_std = df_test['age'].std()

# Comparison
print("=== Custom Functions vs. Pandas Built-in ===")
print(f"Mean - Custom: {custom_mean:.2f} | Pandas: {pandas_mean:.2f}")
print(f"Median - Custom: {custom_median:.2f} | Pandas: {pandas_median:.2f}")
print(f"Std Dev - Custom: {custom_std:.2f} | Pandas: {pandas_std:.2f}")

# Verify they match (allowing for floating-point precision)
assert abs(custom_mean - pandas_mean) < 1e-10
assert abs(custom_median - pandas_median) < 1e-10
assert abs(custom_std - pandas_std) < 1e-10
print("\n✓ All custom functions match Pandas built-in methods!")
```

Question 5: Comprehensive Data Analysis

You are given a dataset of patient ages from a hospital. Your task is to perform a complete analysis, including visualization, summary statistics, and interpretation.

```
# Provided dataset
import pandas as pd
import matplotlib.pyplot as plt

data = {
    'patient_id': range(1, 101),
    'age': [34, 45, 23, 67, 45, 56, 34, 45, 67, 78, 34, 45, 56, 67, 45,
           34, 45, 56, 67, 78, 23, 34, 45, 56, 67, 45, 34, 45, 56, 78,
           89, 34, 45, 56, 67, 78, 45, 34, 45, 56, 67, 89, 34, 45, 56,
           45, 34, 45, 67, 89, 23, 34, 45, 56, 67, 34, 45, 67, 78, 45,
           34, 45, 67, 89, 34, 45, 56, 67, 78, 45, 34, 45, 56, 67, 89,
           45, 34, 56, 67, 78, 45, 34, 56, 67, 89, 34, 45, 67, 78, 45,
           34, 56, 67, 89, 34, 45, 67, 78, 89, 102] # Note: 102 is an outlier
}

df = pd.DataFrame(data)
```

(a) Generate a histogram of the age distribution. Include appropriate labels, title, and grid lines.

```
# Create histogram
plt.figure(figsize=(10, 6))
df['age'].plot.hist(bins=15, edgecolor='black', alpha=0.7, color='steelblue')
plt.title('Distribution of Patient Ages', fontsize=16, fontweight='bold')
plt.xlabel('Age (years)', fontsize=12)
plt.ylabel('Number of Patients', fontsize=12)
```



```
plt.grid(axis='y', alpha=0.3, linestyle='--')
plt.axvline(df['age'].mean(), color='red', linestyle='-', linewidth=2, label=f"Mean: {df['age'].mean():.1f}")
plt.axvline(df['age'].median(), color='green', linestyle='--', linewidth=2, label=f"Median: {df['age'].median():.1f}")
plt.legend()
plt.tight_layout()
plt.show()
```

(b) Calculate and report the mean, median, and standard deviation. Print them in a formatted table.

```
# Calculate statistics
mean_age = df['age'].mean()
median_age = df['age'].median()
std_age = df['age'].std()
min_age = df['age'].min()
max_age = df['age'].max()

# Create formatted table
print("=" * 50)
print("PATIENT AGE SUMMARY STATISTICS")
print("=" * 50)
print(f"{'Statistic':<20} {'Value':>15}")
print("-" * 50)
print(f"{'Mean':<20} {mean_age:>15.2f} years")
print(f"{'Median':<20} {median_age:>15.2f} years")
print(f"{'Standard Deviation':<20} {std_age:>15.2f} years")
print(f"{'Minimum':<20} {min_age:>15.2f} years")
print(f"{'Maximum':<20} {max_age:>15.2f} years")
print("=" * 50)
```

(c) Compare the mean and median. What does their relationship tell you about the distribution? Identify any potential outliers.

Answer: Based on the dataset (which includes an age of 102), the mean will be greater than the median. This indicates right-skewness. The mean is pulled upward by the outlier(s)—in this case, the 102-year-old patient. The median, being resistant to outliers, remains representative of the central tendency of the typical patient. The presence of a 102-year-old in a dataset where most ages cluster in the 30s-70s is a clear outlier that deserves investigation: Is this a data entry error, or a genuine but rare extreme case?

(d) Write code to identify any potential outliers using the “within 3 standard deviations” rule. List any outliers found.

```
# Identify outliers using 3 standard deviations rule
mean = df['age'].mean()
std = df['age'].std()
lower_bound = mean - 3 * std
upper_bound = mean + 3 * std

outliers = df[(df['age'] < lower_bound) | (df['age'] > upper_bound)]

print("=" * 60)
print("OUTLIER DETECTION (3 Standard Deviations Rule)")
print("=" * 60)
print(f"Mean: {mean:.2f} years")
print(f"Standard Deviation: {std:.2f} years")
print(f"Lower bound: {lower_bound:.2f} years")
print(f"Upper bound: {upper_bound:.2f} years")
print("-" * 60)

if len(outliers) > 0:
    print(f"Found {len(outliers)} potential outlier(s):")
    for idx, row in outliers.iterrows():
        print(f"  Patient {row['patient_id']}: {row['age']} years")
else:
    print("No outliers detected using this method.")
print("=" * 60)
```

(e) Write a brief paragraph summarizing your findings for a hospital administrator who is not familiar with statistical terminology.

Answer: “Our patient age data shows that the typical patient is around 50 years old, though we see a wide range from young adults to the elderly. Most patients are between approximately 30 and 70 years old. We did identify one extreme case—a 102-year-old patient—which is unusual and worth verifying for data

accuracy. For resource planning, this tells us we need to be prepared for patients across the adult lifespan, with the highest concentration in middle-aged and older adults. The wide age range means our services should be equipped to handle both younger and older patient needs.”

Part III: Multiple Choice Answer Key and Explanations

Question	Answer
1	B
2	B
3	C
4	D
5	A
6	B
7	C
8	A
9	A
10	D

Detailed Explanations:

- Question 1: Answer: B** The `.plot.hist()` method is specifically designed to generate histograms for quantitative variables. Bar charts (a) are for categorical data, pie charts (c) show proportions, and line plots (d) typically show trends over time.
- Question 2: Answer: B** `pd.cut()` transforms a continuous quantitative variable into discrete bins. This manual binning approach gives the analyst control over bin edges, which is essential when the visualization needs to align with policy-relevant categories (e.g., vaccine eligibility tiers, clinical risk groups).
- Question 3: Answer: C** When the mean exceeds the median, it indicates a right-skewed distribution. The mean is pulled in the direction of the skew by a tail of higher values (older individuals). This is a classic signature of positive skewness.
- Question 4: Answer: D** All three approaches calculate the sample variance. Option (a) uses the built-in Pandas method. Option (b) squares the standard deviation. Option (c) implements the mathematical formula: $\frac{1}{n} \sum (x_i - \bar{x})^2$. All are valid, though (a) is most efficient and readable.
- Question 5: Answer: A** For a roughly normal distribution, the empirical rule states that approximately 68% of data falls within one standard deviation of the mean. Lower bound: $42.5 - 15.2 = 27.3$ years. Upper bound: $42.5 + 15.2 = 57.7$ years.
- Question 6: Answer: B** For $n=4$, the two middle positions are $n//2 - 1 = 1$ (value 20) and $n//2 = 2$ (value 30). The function returns their mean: $(20 + 30) / 2 = 25$.
- Question 7: Answer: C** The presence or absence of gaps is not aesthetic—it carries statistical meaning. Gaps in bar charts reinforce that categories are separate. No gaps in histograms reinforce that the bins are intervals along a continuous number line.
- Question 8: Answer: A** `.value_counts()` returns a Series where the index is unique age values (or bins) and the values are frequencies. Applying `.plot.hist()` to this result would histogram the frequencies, not the original ages. The correct approach is `df['age'].plot.hist()` directly on the raw data.
- Question 9: Answer: A** A single number cannot capture the full story. Two cities could both have a mean age of 38 years—one with all cases tightly clustered between 35-41 years (low variability) and another with cases ranging from infants to the elderly (high variability). These scenarios demand entirely different public health responses. The standard deviation (or another spread measure) is essential.
- Question 10: Answer: D** All three options will run without error and display the standard deviation. However, option (c) uses an f-string for cleaner syntax, formats to two decimal places, and explicitly includes the units (“years”)—making the output more professional and interpretable for reports or presentations.

Part IV: Code Reference Sheet

The following code snippets summarize all essential Pandas operations covered in Lecture 3. Memorize these patterns for the exam.

Loading and Exploring Data

```
import pandas as pd
import matplotlib.pyplot as plt
```



```
# Read CSV
df = pd.read_csv('filename.csv')

# View first few rows
df.head()

# Check column names
df.columns

# Basic info about DataFrame
df.info()
```

Working with a Single Quantitative Variable

```
# Select a column (Series)
ages = df['column_name']

# Summary statistics
mean_val = ages.mean()
median_val = ages.median()
std_val = ages.std()
var_val = ages.var()
min_val = ages.min()
max_val = ages.max()
count_val = ages.count()

# Multiple statistics at once
ages.describe()
```

Visualization

```
# Basic histogram
ages.plot.hist(bins=20, edgecolor='black')
plt.title('Histogram Title')
plt.xlabel('X-axis Label')
plt.ylabel('Frequency')
plt.show()

# Bar chart for binned/categorical data
ages_binned = pd.cut(ages, bins=[0, 18, 35, 50, 65, 100],
                      labels=['0-17', '18-34', '35-49', '50-64', '65+'])
ages_binned.value_counts(sort=False).plot.bar()
plt.show()
```

Manual Binning with `pd.cut()`

```
# Create bins
df['age_group'] = pd.cut(df['age'],
                        bins=[0, 18, 35, 50, 65, 120],
                        labels=['0-17', '18-34', '35-49', '50-64', '65+'],
                        right=False) # right=False means bins are [low, high)

# Count observations in each bin
bin_counts = df['age_group'].value_counts(sort=False)

# Plot
bin_counts.plot.bar()
```

Custom Statistical Functions (First Principles)

```
def mean_custom(data):
    return sum(data) / len(data)

def median_custom(data):
    sorted_data = sorted(data)
    n = len(sorted_data)
    if n % 2 == 1:
        return sorted_data[n // 2]
    else:
        return (sorted_data[n // 2 - 1] + sorted_data[n // 2]) / 2

def std_custom(data):
    mean = mean_custom(data)
    squared_deviations = sum((x - mean) ** 2 for x in data)
    variance = squared_deviations / (len(data) - 1) # sample variance
    return variance ** 0.5
```

Formatting Output

```
# f-string formatting
print(f"Mean: {ages.mean():.2f} years")
print(f"Median: {ages.median():.2f} years")

# Table formatting
print("=" * 40)
print(f"{'Statistic':<20} {'Value':>15}")
print("-" * 40)
print(f"{'Mean':<20} {ages.mean():>15.2f}")
print(f"{'Std Dev':<20} {ages.std():>15.2f}")
print("=" * 40)
```